Scaling AI

Day 1: Parallel Computing Foundations

Real-world systems • technical depth • hands-on benchmarking

Big idea

Parallelism is a design choice: faster only when useful compute beats overhead.

Student output

Benchmark serial, threaded, process-based, and vectorized approaches.

Today’s Roadmap

A visual path from motivation to measurable parallel AI

01 **Motivation**

Why AI becomes a systems problem

02 **Mental models**

Concurrency, parallelism, distributed computing

03 **Real systems**

Search, recommender, LLM serving, image pipelines

04 **Technical core**

GIL, processes, overhead, load balance, races

05 **Lab**

Benchmark, explain, predict

Learning Objectives

What students should be able to do by the end of Day 1

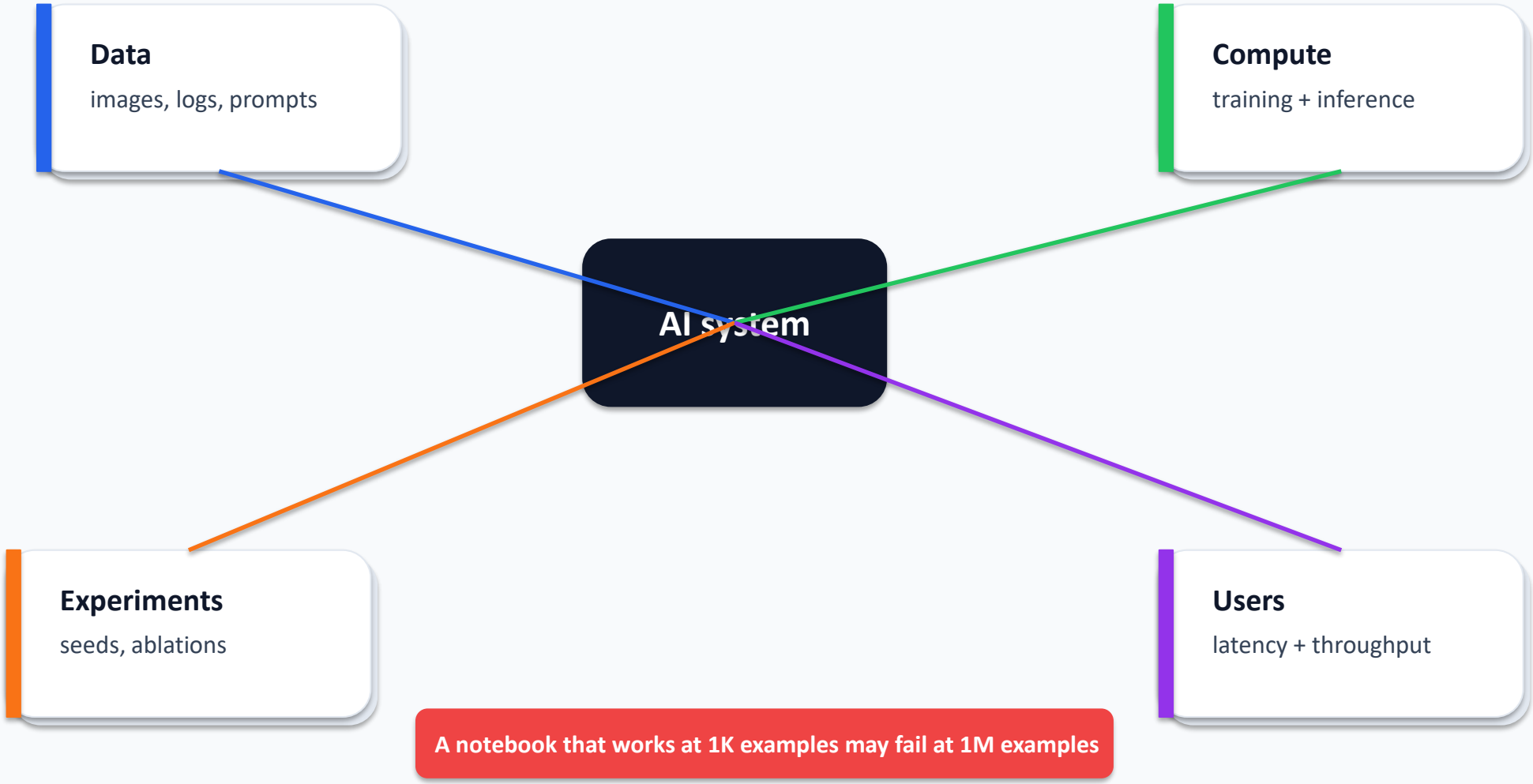
1. **Explain**
   why AI workloads need scaling
2. **Distinguish**
   threading, multiprocessing, distributed computing
3. **Identify**
   data, task, and pipeline parallelism
4. **Analyze**
   GIL, overhead, load balance, speedup limits
5. **Measure**
   runtime, speedup, efficiency, latency, throughput
6. **Connect**
   concepts to the lab

1. Why Scaling AI Matters

From model scripts to full systems

AI Workloads Grow in Many Dimensions

The challenge is not just “train a bigger model”



Discussion Warm-Up

Find what can run at the same time

A

Scenario A: Image classifier

5,000 images worked. Now 500,000 images make preprocessing take hours.

Ask: load, decode, resize, augment — which steps are independent?

B

Scenario B: Recommender

20,000 users need recommendations overnight.

Ask: can user scoring be split across workers? What must be gathered?

C

Scenario C: AI assistant

Many prompts arrive. Each prompt is slow but independent.

Ask: how do queueing and batching affect latency?

Questions: What runs now? What waits? What must be combined?

2. Mental Models

Vocabulary and patterns students will reuse all week

Core Vocabulary

Terms that separate “looks parallel” from “is faster”

↔

Concurrency

Multiple tasks overlap in time; useful even on one core.

∥

Parallelism

Multiple computations actually run at the same time.

T

Multithreading

Threads share memory inside one process.

P

Multiprocessing

Separate processes, separate memory spaces.

D

Distributed

Processes/machines communicate by messages.

B

Baseline

Simplest correct serial version for comparison.

Analogy: One Kitchen vs Many Kitchens

The intuition behind scheduling, bottlenecks, and communication

1

Serial

One chef, one recipe step at a time.

↻

Concurrent

One chef switches while waiting for water to boil.

‖

Parallel

Multiple chefs work at once in one kitchen.

⇄

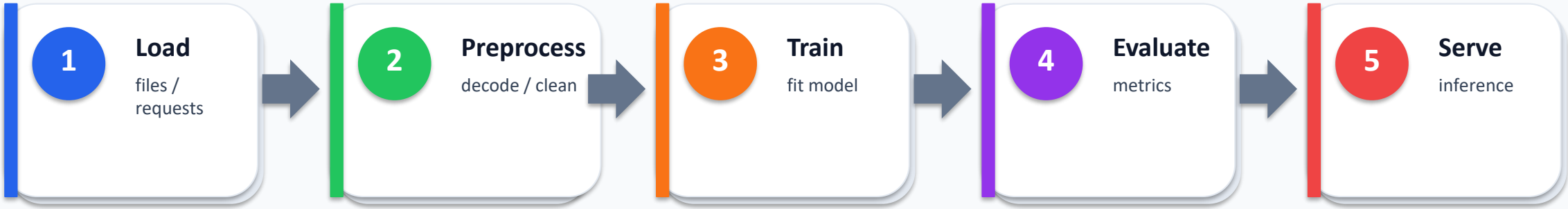
Distributed

Multiple kitchens coordinate and combine orders.

Too many workers can create contention — overhead, locks, memory bandwidth, communication

AI Pipeline: Where Can We Parallelize?

Look for independent chunks and expensive repeated work



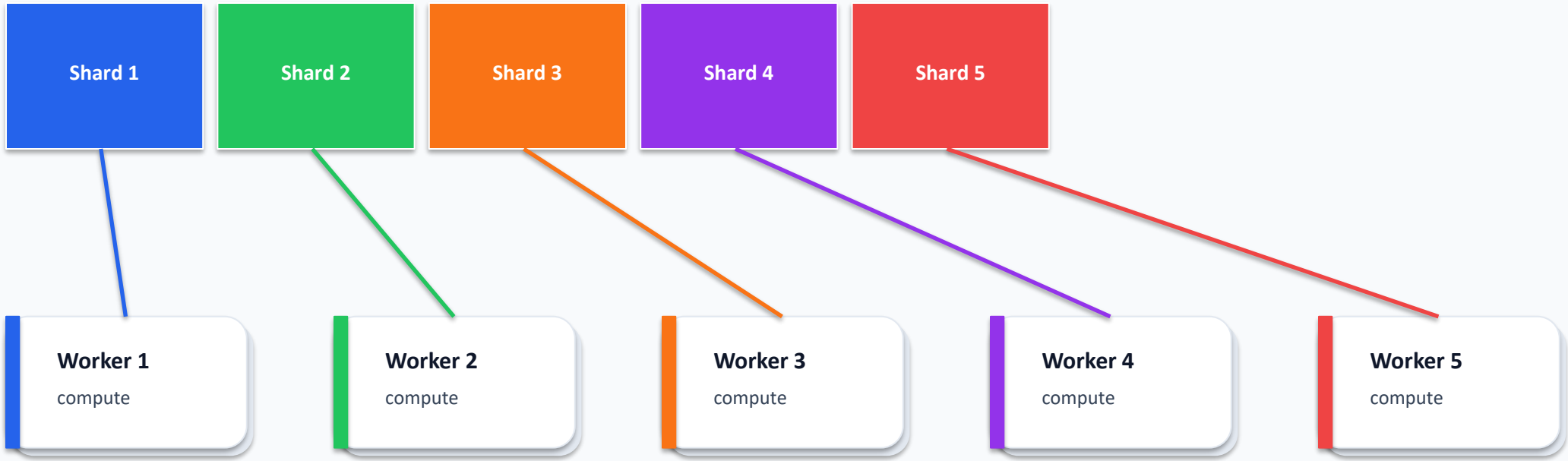
Data parallelism
same operation, different examples

Task parallelism
different independent jobs

Pipeline parallelism
overlap workflow stages

Data Parallelism

Same computation, different shards



Best when examples are independent and aggregation is small

Task and Pipeline Parallelism

Different ways to overlap useful work

TP

Task parallelism

Run different independent jobs: hyperparameter trials, checkpoint evaluation, embedding generation, separate model variants.

Risk: tasks compete for CPU, RAM, disk, or GPU.

PP

Pipeline parallelism

Split a workflow into stages: read → decode → augment → batch → infer.

Risk: bottleneck stage controls throughput; queues can grow.

Hardware Mental Model

Performance often depends on the slowest resource

CPU

CPU cores

Flexible workers for orchestration and data tasks.

GPU

GPU

Massive tensor arithmetic for neural networks.

RAM

Memory

Hidden bottleneck for data-heavy operations.

I/O

Disk / network

Usually limits loading and distributed data movement.

NET

Cluster

Many machines; communication is a first-class cost.

Python Tools Preview

Choose tools by bottleneck, then measure

I/O

I/O-bound?

Use threading or ThreadPoolExecutor.

CPU

CPU-bound Python?

Use multiprocessing or ProcessPoolExecutor.

vec

Numeric arrays?

Try NumPy/PyTorch vectorization first.

waiting?

calculating?

matrix ops?

Rule of thumb: baseline → benchmark → optimize → explain

3. Real-World Systems

Concrete examples students can recognize

Real-World Examples of Parallel AI Workloads

Each example maps to a final project strategy

Search indexing

Map shards → emit tokens → reduce counts

★

Recommendation

Score users/items → gather top-k

LLM

LLM inference

Queue prompts → batch → route to workers

IMG

Image pipelines

Decode → augment → batch images

∇

Distributed training

Replicas → local gradients → all-reduce

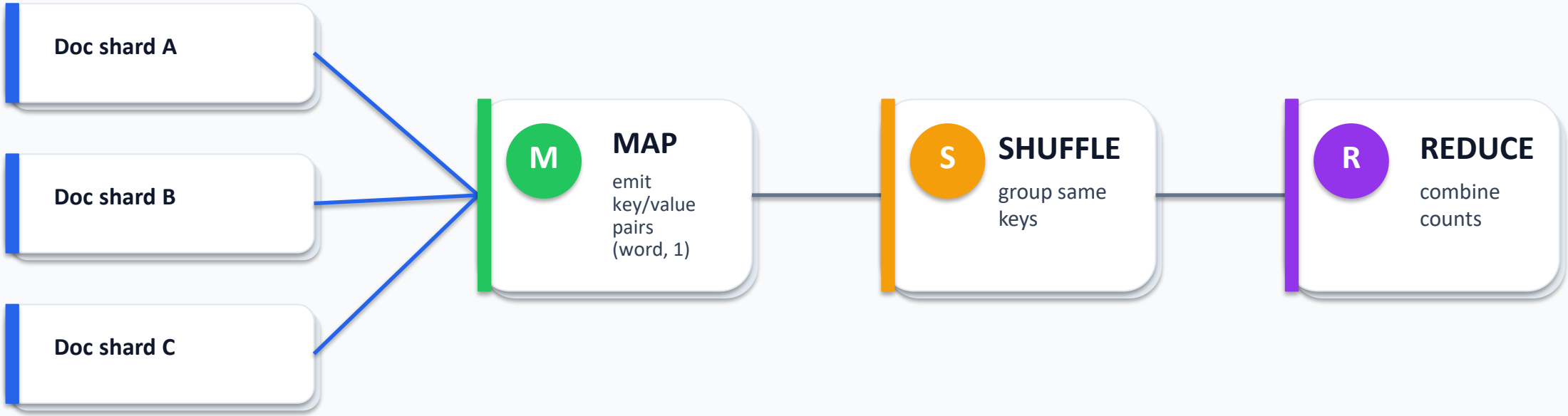
HP

Hyperparameter search

Independent trials → aggregate leaderboard

Case Study: Search Indexing as MapReduce

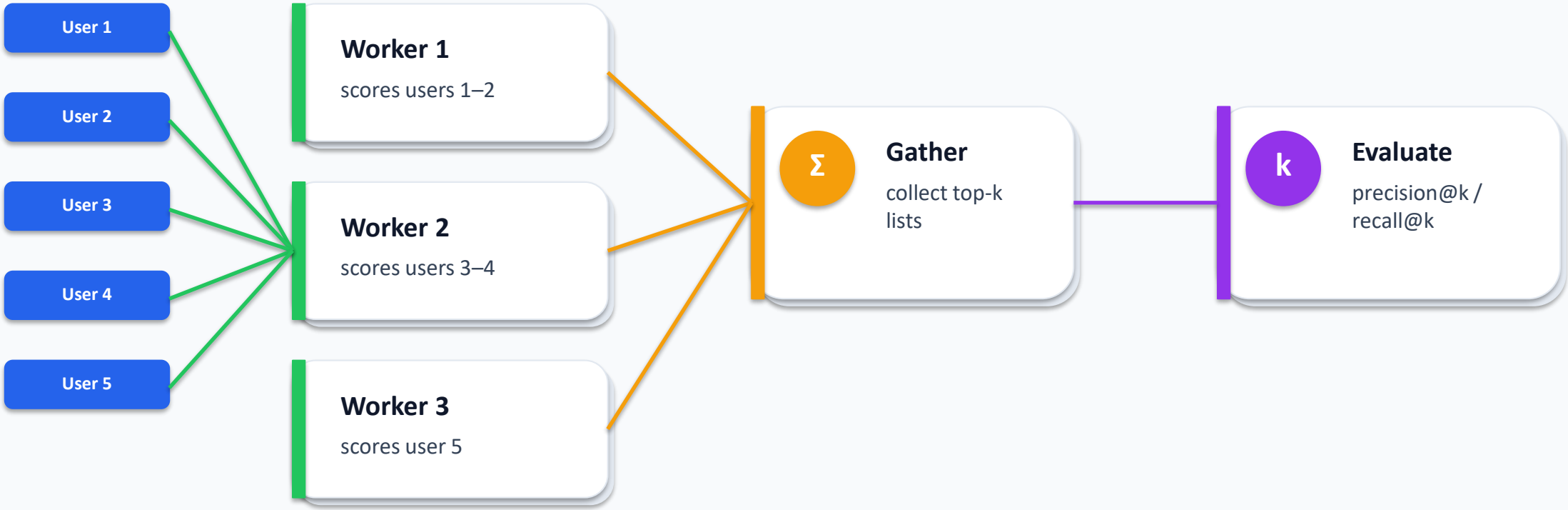
A classic pattern behind large-scale data processing



Many AI/data tasks are easier to scale when expressed as map → shuffle → reduce

Case Study: Recommendation at Scale

A clean example of embarrassingly parallel scoring



Bottlenecks: memory bandwidth, candidate size, matrix operations, top-k sorting

Case Study: Image Classifier Pipeline

Sometimes the model waits for the data pipeline

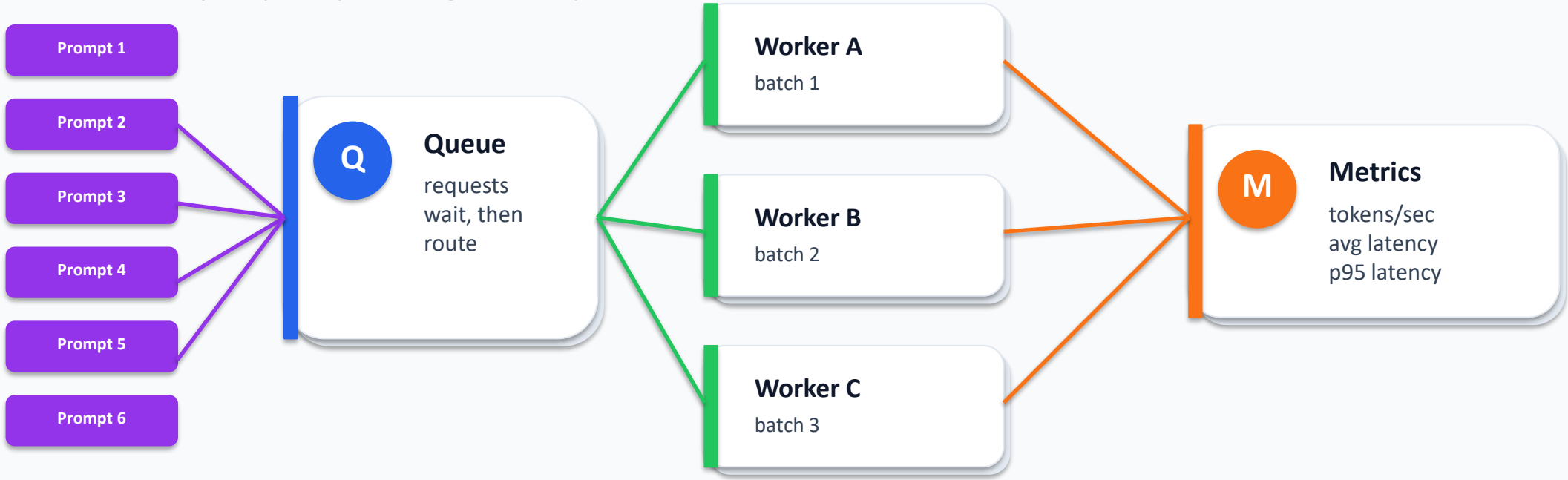
Serial load

GPU/CPU idle waiting for batch

Parallel loader

Case Study: LLM Serving — Throughput vs Latency

Parallelism is not just speedup; it changes user experience



Batching often improves throughput but can increase per-user waiting time

4. Deeper Technical Content

What really determines whether parallelism helps

Deeper Concept: Python’s GIL

Why threads may not speed up CPU-heavy Python code

CPU

CPU-bound Python

Threads compete for the interpreter lock; parallel speedup is limited.

I/O

I/O-bound work

Threads can help while tasks wait on files/network.

P

Processes

Separate interpreters can use multiple cores.

vec

Vectorized code

NumPy/PyTorch can run optimized native kernels.

Rule: CPU-heavy pure Python → processes; waiting → threads; arrays → vectorize first

Threads vs Processes: Memory and Communication

Parallel design is also data-movement design

T

Threads

Shared memory makes communication easy, but shared mutable state can create race conditions. Thread synchronization can add overhead.

P

Processes

Separate memory enables true CPU parallelism, but Python objects must be serialized/pickled between processes.

Design implication: send compact task descriptions; avoid repeatedly moving huge arrays

Simple Performance Model

The slowest worker and communication overhead matter

```
Total parallel time ≈

  setup cost
+ scheduling cost
+ max(worker compute times)
+ communication / serialization cost
+ final aggregation cost

Speedup = serial_time / parallel_time
```

max

Slowest worker controls finish time

max(worker compute times)

⇄

Communication is real work

data must move between workers

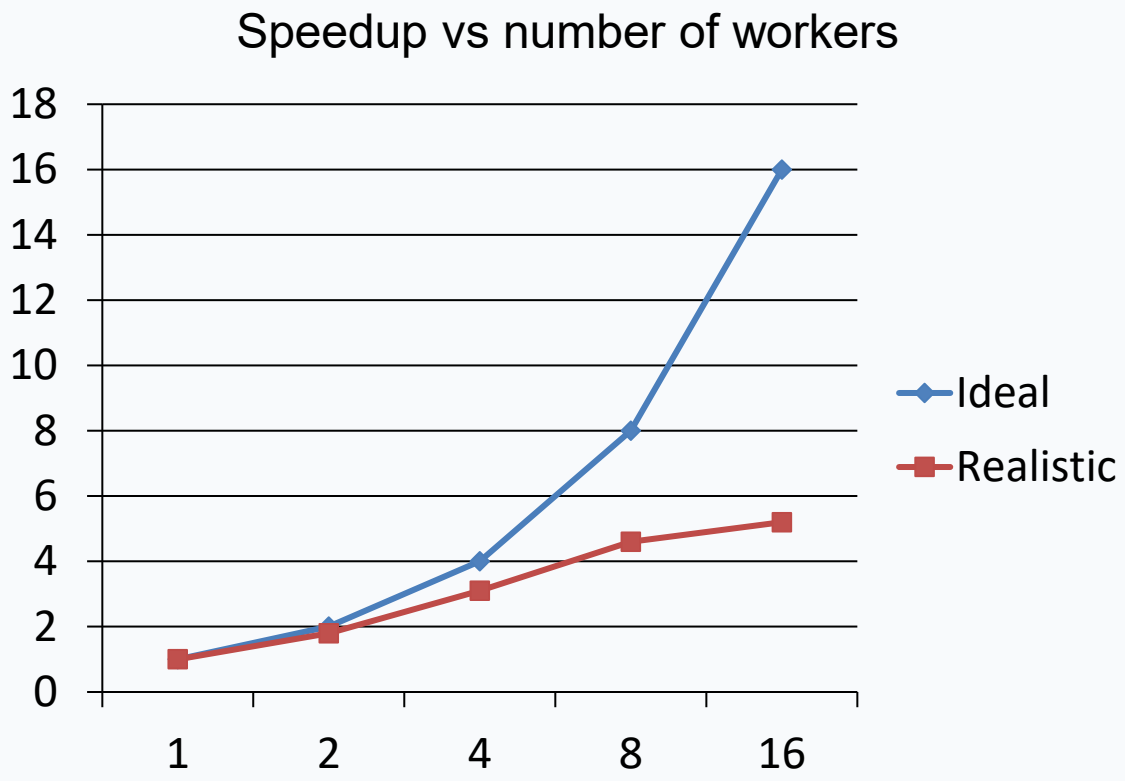
✓

Useful compute must dominate

otherwise parallelism loses

Speedup and Efficiency

Measure with a baseline, not intuition



S **Speedup**

baseline runtime / parallel runtime

E **Efficiency**

speedup / number of workers

T/L **Throughput vs latency**

items/sec vs time per item

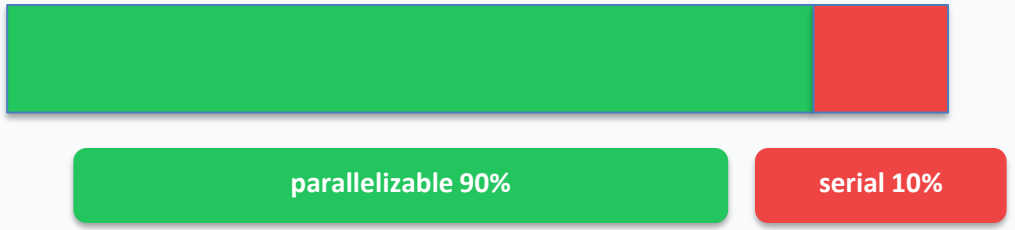
Amdahl’s Law: Bottleneck Intuition

The serial part caps total speedup

```
Speedup(N) = 1 / ((1 - P) + P/N)

P = parallelizable fraction
N = number of workers

Example:
P = 0.90, N = 8
Speedup ≈ 4.7x, not 8x
```

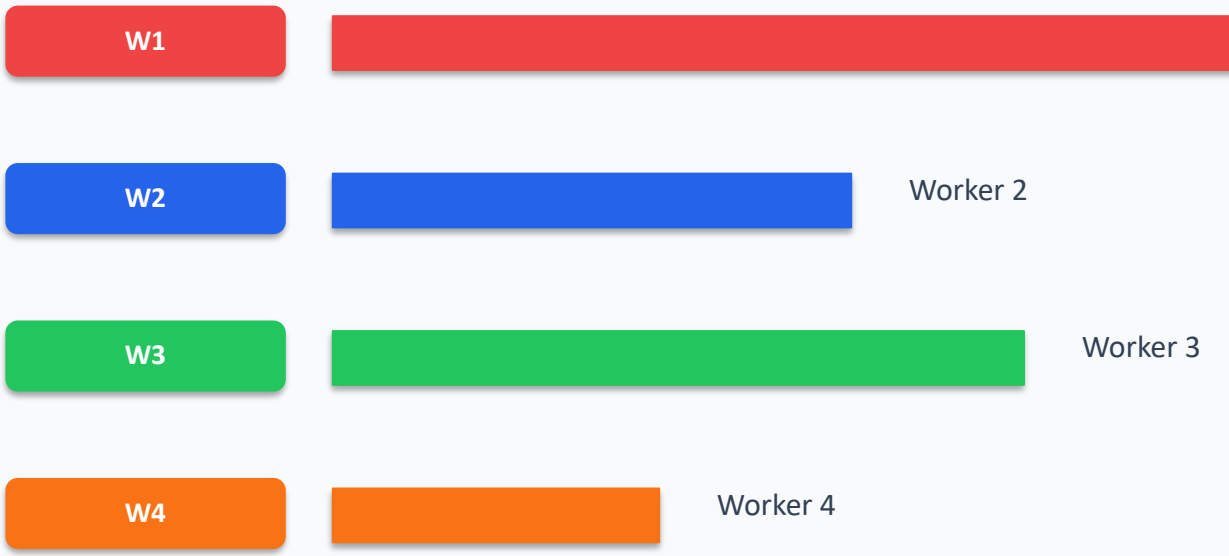


Lesson

Optimize the biggest bottleneck first; adding workers cannot remove serial work.

Load Balance: The Slowest Worker Wins

Uneven tasks reduce speedup



Static partitioning

Simple but can leave workers idle.

Dynamic scheduling

Better balance, more scheduling overhead.

Chunk Size: Too Small vs Too Large

One knob can change performance dramatically

tiny

Too small

Scheduling and communication dominate. Workers spend time receiving tiny tasks.

huge

Too large

Poor load balance. Some workers finish early and sit idle.

✓

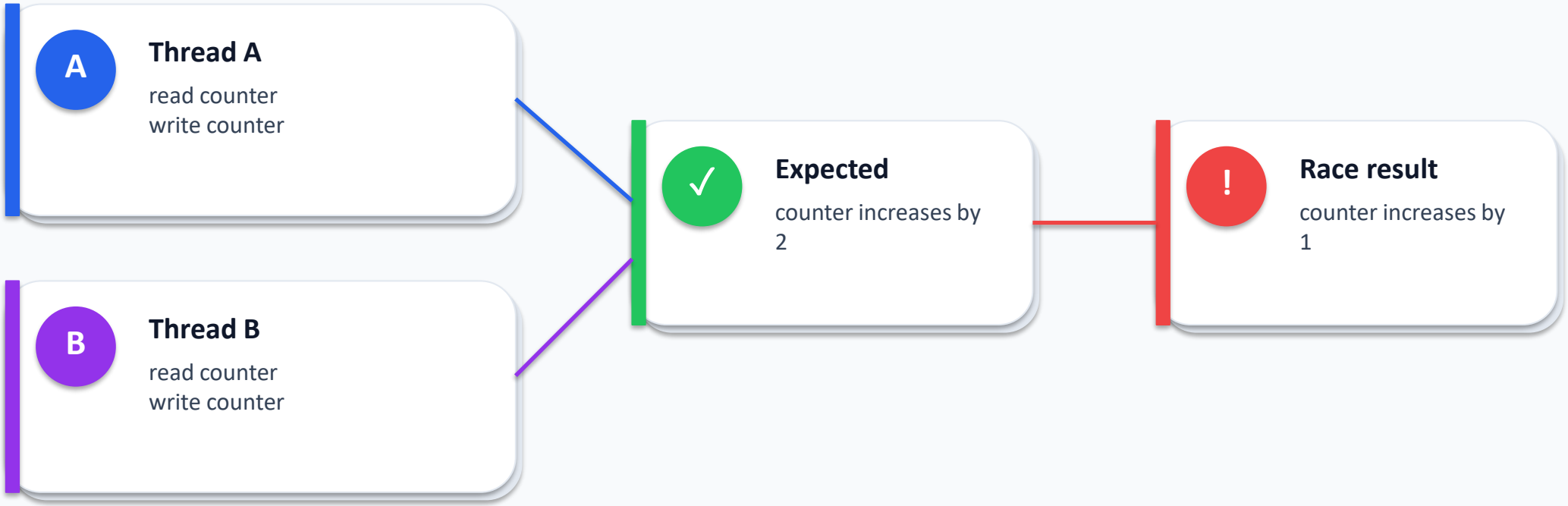
Just right

Enough work per chunk to amortize overhead, enough chunks for balancing.

Lab extension: vary chunksize and explain the performance curve

Correctness Risk: Race Conditions

Parallel code can be fast and wrong



Fixes: avoid shared mutable state, use locks/queues, prefer map → reduce aggregation

Preview: MPI-Style Communication Patterns

These patterns return on Day 5 and Day 6

1. **Broadcast**
   one → all
2. **Scatter**
   split data
3. **Gather**
   collect results
4. **Reduce**
   combine values
5. **All-reduce**
   combine + share

AI connection: distributed training uses all-reduce-style gradient aggregation

Preview: Data-Parallel Training Internals

A first-principles view before using real DDP

Same optimizer step on every replica keeps models synchronized

5. Lab Connection

Benchmark first, optimize with evidence

Visual section

Lab Overview: Serial vs Parallel Benchmark

Observe first; explain second

1

1. Serial

plain Python baseline

T

2. Threads

ThreadPoolExecutor

P

3. Processes

ProcessPoolExecutor

V

4. Vectorized

NumPy array operations

Benchmark CPU-heavy and I/O-like tasks; create runtime, speedup, and efficiency tables

Starter Code: Timing Helper

Use reliable wall-clock timing for first measurements

```
import time
from contextlib import contextmanager

@contextmanager
def timer(label):
    start = time.perf_counter()
    yield
    end = time.perf_counter()
    print(f"{label}: {end - start:.4f} seconds")

def cpu_task(x):
    # intentionally CPU-heavy pure Python work
    total = 0
    for i in range(5000):
        total += (x * i) % 97
    return total
```

Starter Code: Compare Execution Models

Same function, same data, different execution model

```
from concurrent.futures import ThreadPoolExecutor, ProcessPoolExecutor

data = list(range(20_000))

with timer("serial"):
    y1 = [cpu_task(x) for x in data]

with timer("threads: 4 workers"):
    with ThreadPoolExecutor(max_workers=4) as ex:
        y2 = list(ex.map(cpu_task, data))

with timer("processes: 4 workers"):
    with ProcessPoolExecutor(max_workers=4) as ex:
        y3 = list(ex.map(cpu_task, data, chunksize=100))

assert y1 == y2 == y3
```

Expanded Lab: Add an I/O-Bound Benchmark

Threads should help when the program is mostly waiting

```
from concurrent.futures import ThreadPoolExecutor
import time

def io_like_task(x):
    # Simulates waiting on file/network I/O
    time.sleep(0.01)
    return x * x

data = range(200)

for workers in [1, 2, 4, 8, 16]:
    start = time.perf_counter()
    with ThreadPoolExecutor(max_workers=workers) as ex:
        results = list(ex.map(io_like_task, data))
    elapsed = time.perf_counter() - start
    print(workers, elapsed)
```

Mini Challenge

Turn measurements into an explanation

1

Run

CPU-heavy + I/O-like benchmarks with 1, 2, 4, 8, 16 workers.

2

Record

runtime, speedup, efficiency, correctness.

3

Explain

threads vs processes, overhead, chunksize, bottlenecks.

Optional: add NumPy vectorization and compare against Python-level parallelism

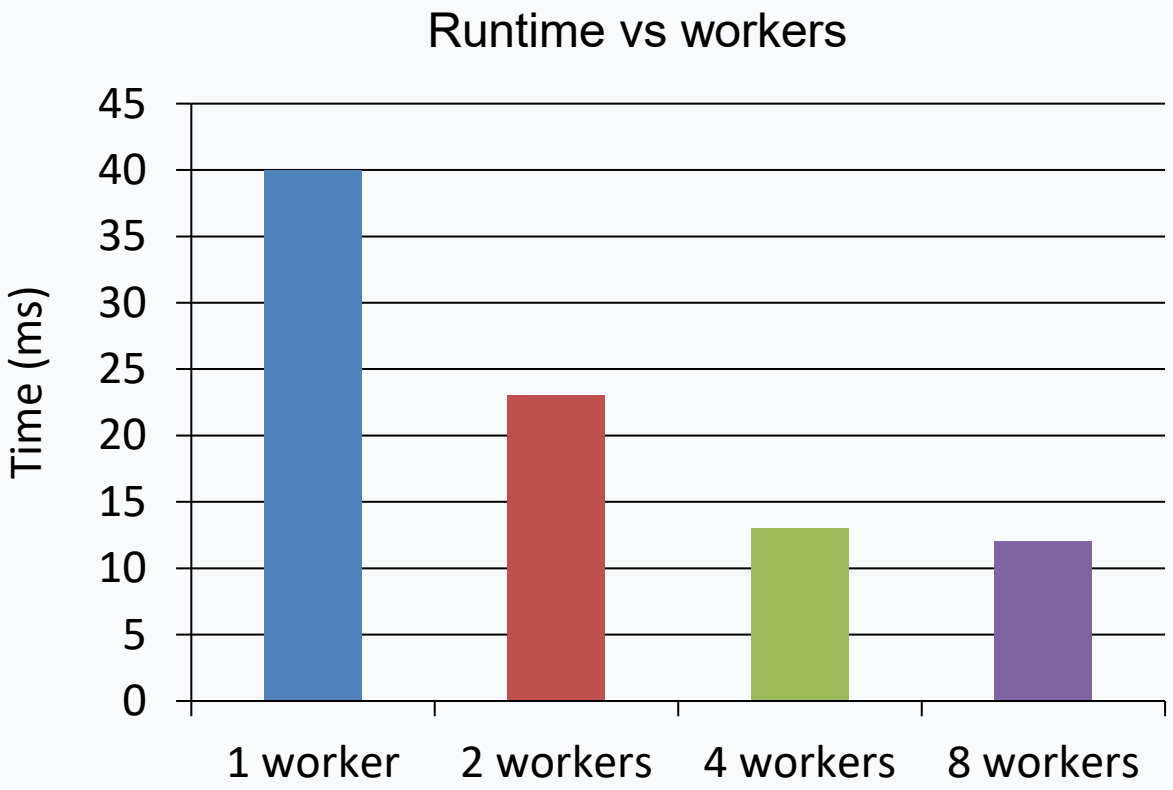
Benchmarking Rules

Good measurements are part of good science

- ✓ Run each experiment multiple times
- ✓ Use same input across versions
- ✓ Separate setup from processing when possible
- ✓ Record machine information
- ✓ Check correctness, not only speed
- ✓ Report surprising results honestly

Example Result Format

The chart is the start of the conversation, not the end



Interpretation question

Why did 8 workers barely improve over 4?

Possible reasons

overhead, memory bandwidth, task size, disk bottleneck, load imbalance

Next step

profile the bottleneck and redesign the chunks

Connect to Final Projects

Each team needs a baseline and a parallel version

1. **Image classifier**
   parallel loading, augmentation, inference

2. **Recommendation**
   parallel score/top-k computation

3. **Data analysis**
   chunk processing + reduce

4. **LLM simulator**
   prompt queue + worker pool

5. **Hyperparameter search**
   parallel trials + leaderboard

6. **Federated simulation**
   local training + weight averaging

Project Design Worksheet: Bottleneck → Strategy

Use this before writing parallel code

1

Slowest step?

data load, preprocessing, training, inference, evaluation

2

Independent work?

examples, users, files, prompts, trials

3

Bottleneck type?

CPU, I/O, GPU, memory, communication

4

Communication?

what data must move between workers

5

Baseline?

what serial version proves improvement

6

Metric?

runtime, throughput, latency, accuracy, cost

Exit Ticket

Short written reflection + benchmark notebook

Q

Answer in 3–5 sentences

- Concurrency vs parallelism?
- Why processes for CPU-heavy Python?
- Why can parallelism be slower than serial?
- What bottleneck might remain?
- Where could your project use data parallelism?

✓

Submit

- Benchmark notebook
- Timing table
- Speedup calculation
- One bottleneck hypothesis
- One project-parallelization idea

References and Further Reading

For students who want to go deeper

1. **Python documentation: concurrent execution, threading, multiprocessing, and the GIL.**

2. **Google MapReduce paper: simplified data processing on large clusters.**

3. **MPI tutorials: broadcast, scatter, gather, reduce, and all-reduce.**

4. **PyTorch DistributedDataParallel tutorial and API reference.**

5. **Optional advanced tools: cProfile, line_profiler, Scalene, PyTorch Profiler.**